

Discovery Executive Summary

Project: discovery-24jul-003 · Generated: 24/07/2026, 15:51:28

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 1 discovery analysis run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating	Hotspot Score
1	Architecture & Design Analysis	HIGH RISK	—

1. Architecture & Design Analysis

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk Missing Service Layer (H2), Missing Repository Pattern (H3), Shared Database Coupling (H9), and a fully Duplicated Parallel Backend (H10).

EXECUTIVE SUMMARY

FSDKC is a small but structurally revealing full-stack "Klearcom" voice-observability platform spanning three layers: a PHP/Laravel API (`backend/`), a React/TypeScript SPA (`frontend/`), and a parallel Node/Express mock API (`dev-api/`) that re-implements the entire backend surface. Absolute file sizes are modest, so no God Class or oversized-component threshold is breached, but the design-level hotspots are severe: there is **no repository layer at all** (0 repository classes against ~38 direct Eloquent/ORM access points) and **no application-service tier** for read paths, so controllers such as `DashboardController` and `LegacyReportController` carry KPI math, `extract()`-based mapping, and cross-domain queries directly. The dominant risk is **change amplification through duplication and shared data**: the reachability formula is copy-pasted across `ConnectController`, `RealTimeTestService`, `LegacyReportController`, and `dev-api/realtime.js`, `buildTree` across three files, and both the Laravel and Express backends read/write the *same* MongoDB `klearcom` collections with no ownership. The Connect and Discovery bounded contexts (declared in `AGENTS.md` module files) are violated by controllers and services that touch both domains at once. Layers covered: **backend** (6 Laravel controllers, 2 services, 4 models, 1 legacy mapper), **frontend** (9 React components/pages, 1 hook, 1 store), and a **second**

backend (Express dev-api, ~17 route handlers). The overall verdict is High Risk, driven by the missing service/repository tiers, shared-database coupling, and a fully duplicated parallel backend.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Fat Controllers	Avg LOC per controller (+ business logic)	<150	150–300	>300	74 LOC avg, but 3/6 hold business logic	MODERATE
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	~26 direct access points	HIGH RISK
H3	Missing Repository Pattern	Direct DB/ORM access points	<10	10–20	>20	~38 access points, 0 repositories	HIGH RISK
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	GOOD
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	2 (<code>LegacyDataMapper</code> , <code>dev-api/store.js</code>)	MODERATE
H6	Direct SQL in Controllers	ORM/query-builder compliance %	>90%	60–90%	<60%	~85% (2 in-controller query builders)	MODERATE
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0 (largest 276 LOC)	GOOD
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	5	MODERATE
H9	Shared Database Coupling	Data stores shared across domains	<10%	10–30%	>30%	~37% (3/8 stores, 2 backends)	HIGH RISK
H10	Duplicated Parallel Backend (<i>additional</i>)	Business algorithms duplicated across backends	0	1–2	>2	3 (reachability, buildTree, KPI math)	HIGH RISK

F1	Business Logic in Components	Avg LOC per component (+ logic)	<150	150–300	>300	81 LOC avg, 3 carry orchestration logic	MODERATE
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	6 files build endpoints inline	MODERATE
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0 (largest 222 LOC)	GOOD
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	≤2 levels	GOOD
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	2 (class component + manual-fetch widget)	MODERATE

1.4 Actions Required

Hotspot	Action	Rating	Priority
H2 Missing Service Layer	Introduce <code>ConnectService</code> / <code>DiscoveryService</code> / <code>DashboardMetricsService</code> ; move all controller model queries into them	HIGH RISK	CRITICAL
H3 Missing Repository Pattern	Add repository interfaces + Eloquent impls, bound in <code>AppServiceProvider</code> ; encapsulate the "recent 20 checks" query	HIGH RISK	CRITICAL
H9 Shared Database Coupling	Give each context its own Mongo collections; add an ACL over the dev-api; adopt versioned migrations	HIGH RISK	HIGH
H10 Duplicated Parallel Backend	Consolidate reachability/ <code>buildTree</code> /KPI math into single shared services; add cross-backend contract test	HIGH RISK	HIGH
H1 Fat Controllers	Extract KPI/reachability/mapping logic out of <code>Dashboard</code> / <code>Legacy</code> / <code>Connect</code> controllers into services	MODERATE	HIGH
H8 Domain Boundary Violations	Split <code>RealTimeTestService</code> and <code>LegacyReportController</code> per bounded context; consume via published interfaces	MODERATE	HIGH
H5 Shared Utility Abuse	Replace <code>LegacyDataMapper</code> <code>extract()</code> with DTOs; extract <code>buildTree</code> from <code>store.js</code>	MODERATE	MEDIUM
H6 Direct SQL in Controllers	Move <code>LegacyReportController</code> and dev-api diagnostics query builders into repositories	MODERATE	MEDIUM

F1 Business Logic in Components	Move post-run invalidation into a hook; migrate <code>LegacyDashboardWidget</code> off manual fetching	MODERATE	MEDIUM
F2 Missing Frontend Service/Data Layer	Add typed per-domain API service modules; remove inline endpoint strings from 6 files	MODERATE	MEDIUM
F5 Legacy / Inconsistent Component Patterns	Convert <code>LegacyMonitorPoller</code> to a function component with cleanup; add an Error Boundary; standardize on React Query	MODERATE	MEDIUM

1.5 Expected Outcomes

- **Testable business rules:** reachability, IVR-tree, and KPI logic move into services/repositories that can be unit-tested without booting HTTP or the datastore, extending the coverage that today stops at `ReachabilityCalculationTest`.
- **Single source of truth:** consolidating the duplicated reachability/ `buildTree` /KPI algorithms (H1/H5/H10) means a rule change is a one-file edit, eliminating dashboard-vs-report and prod-vs-dev divergence.
- **Independently evolvable domains:** enforcing the Connect and Discovery bounded contexts and giving each its own data ownership (H8/H9) makes either domain extractable into its own service without untangling shared models or collections.
- **Swappable persistence:** a repository tier (H3) decouples business logic from Eloquent/Mongo, enabling schema changes and datastore swaps behind stable interfaces.
- **Resilient, consistent frontend:** a typed per-domain service layer plus an Error Boundary and function-component conventions (F1/F2/F5) remove inline endpoint duplication, stop interval leaks, and prevent a single failed fetch from crashing the SPA.